



MARIO'S GARDEN

Gizmo - Group 43

Zoe Jung

Arancha Ramirez

05/12/2023

Game Algorithm Structure

This algorithm displays the outline of the project, including the elements, main logic, and user interactions.

Master Game Start

Pressing both buttons starts the Master Game.
All variables are reset to base conditions for the new game.

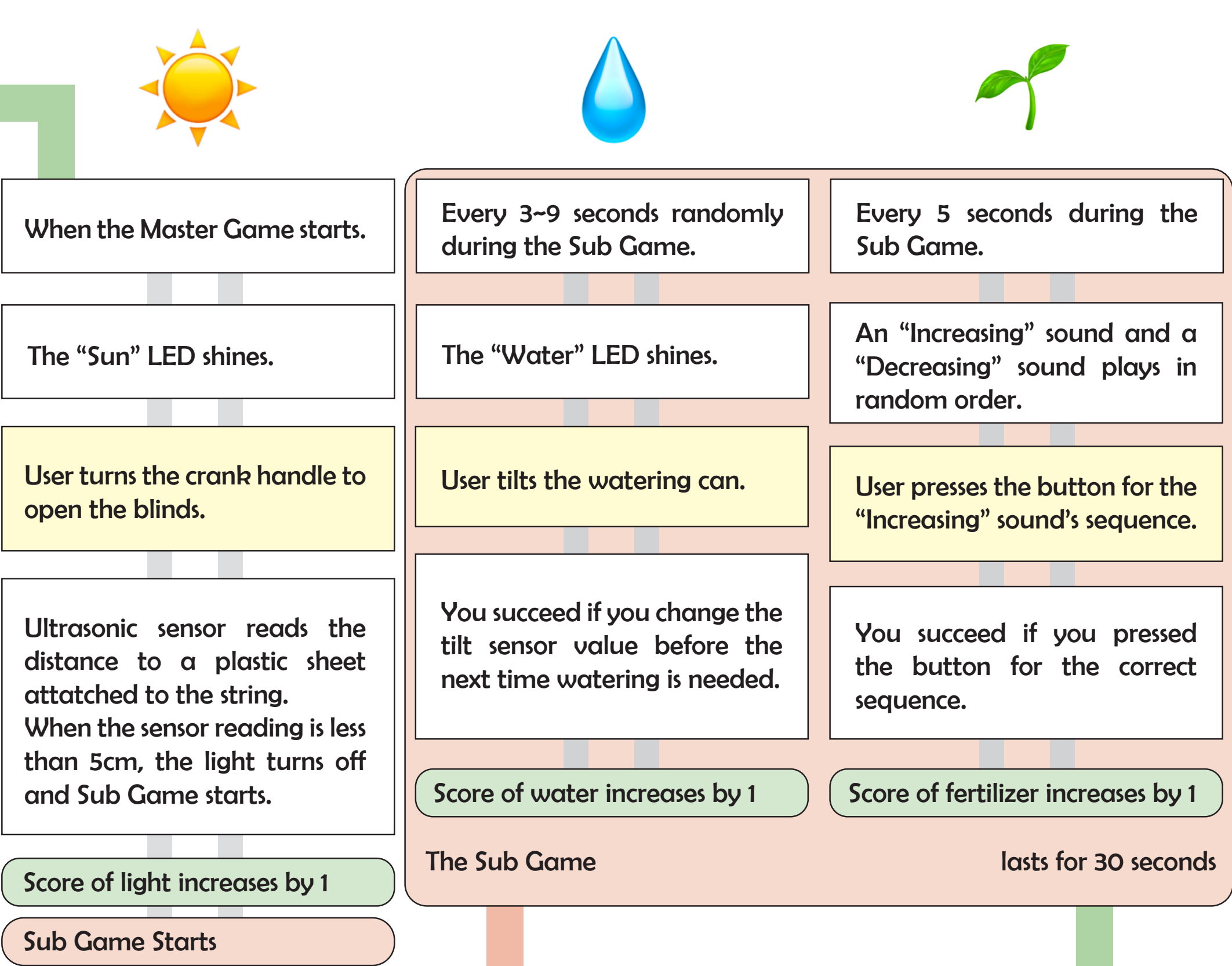
The Back Story

There is one thing we have in comon: We are very bad at growing plants.
So when we spotted the plant market at Imperial, we agreed about how we only keep fake plants and not live ones, since we find it difficult to keep plants alive. The biggest reason of such difficulty is responding to the plant's needs at the right time. Like watering it before it gets dehydrated or letting sunlight in every day.
This game aims to provide great practice of reacting to the plant's needs.

The flower's height is adjusted according to the score change of all 3 elements.

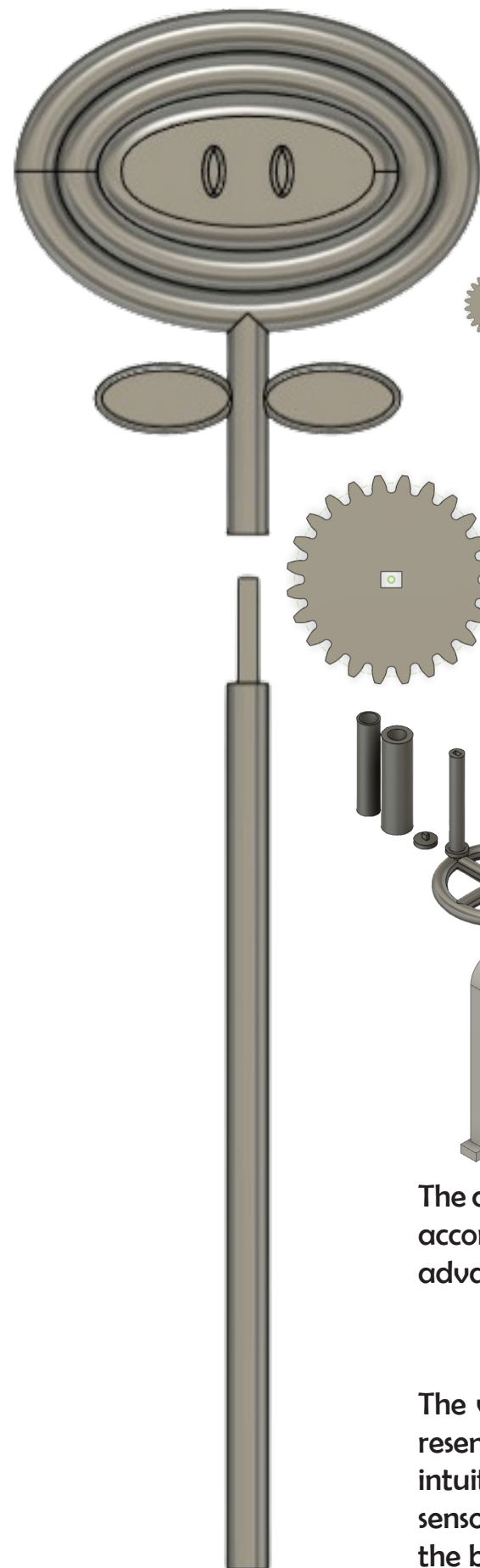
Master Game Finish

The Master Game ends with the Sub Game, which lasts for 30 seconds.
All lights and motors are turned off.



The Physical Components

3D Printing



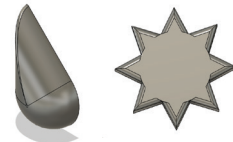
The Motor Gear



A gear is fitted to the DC motor. This acts as a pinion to the rack (the stem of the flower).



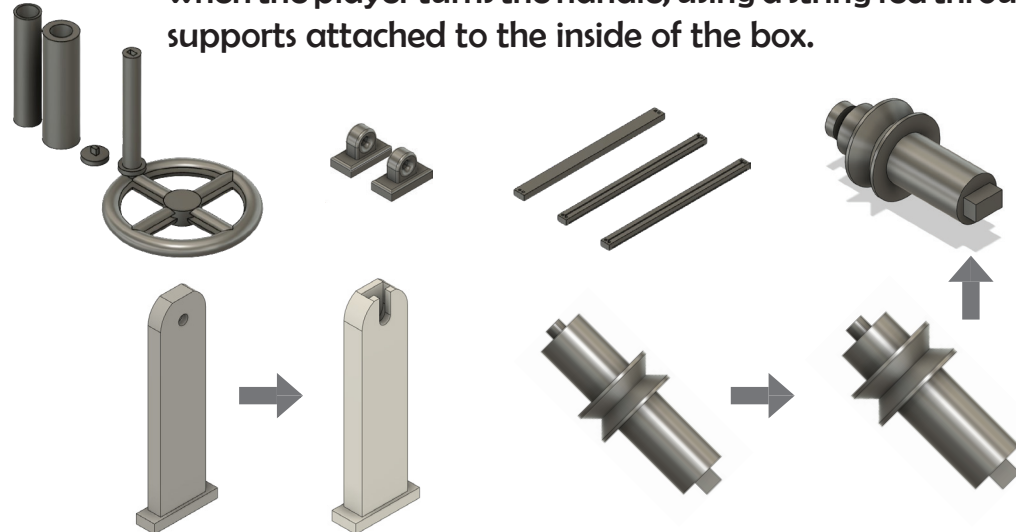
The LED covers



The water drop and sun covers went through several design changes for them to be thin enough to let light through, but thick enough to withstand removal of supports.

The Crank Handle System

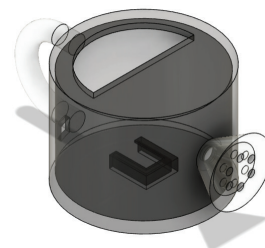
Crank handle pulls the blinds up to let 'sunlight' through when the player turns the handle, using a string fed through supports attached to the inside of the box.



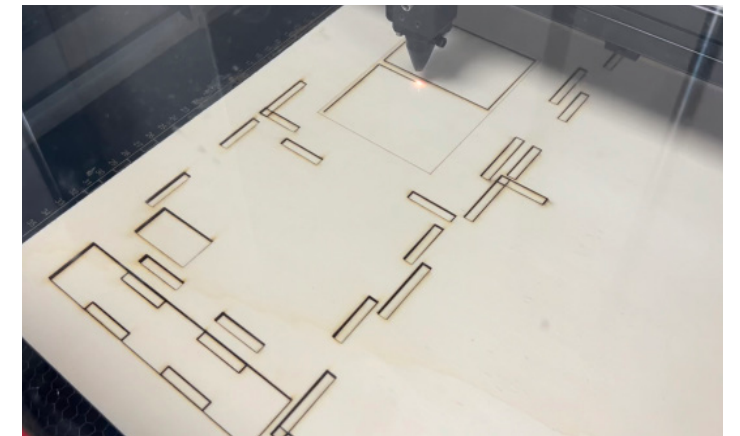
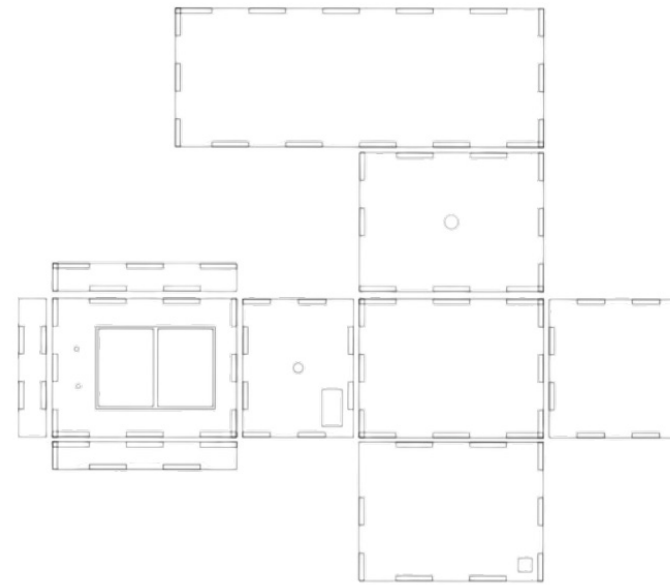
The crank handle supports designs went through many changes according to the adjustments of other components, and further advances for increased stability.

The Watering Can

The watering can was modelled to resemble a watering can to allow intuitive interactions, with a tilt sensor housing inside, and a hole on the back for the wires.



Laser Cutting



The box's dimensions and layout was initially sketched, modeled in 3D, and then translated to Illustrator so that it could be laser cut and assembled

The Final Model Assembly Process

Step 1: The lasercut box was assembled, and glued except for the parts that allow access to the inside.

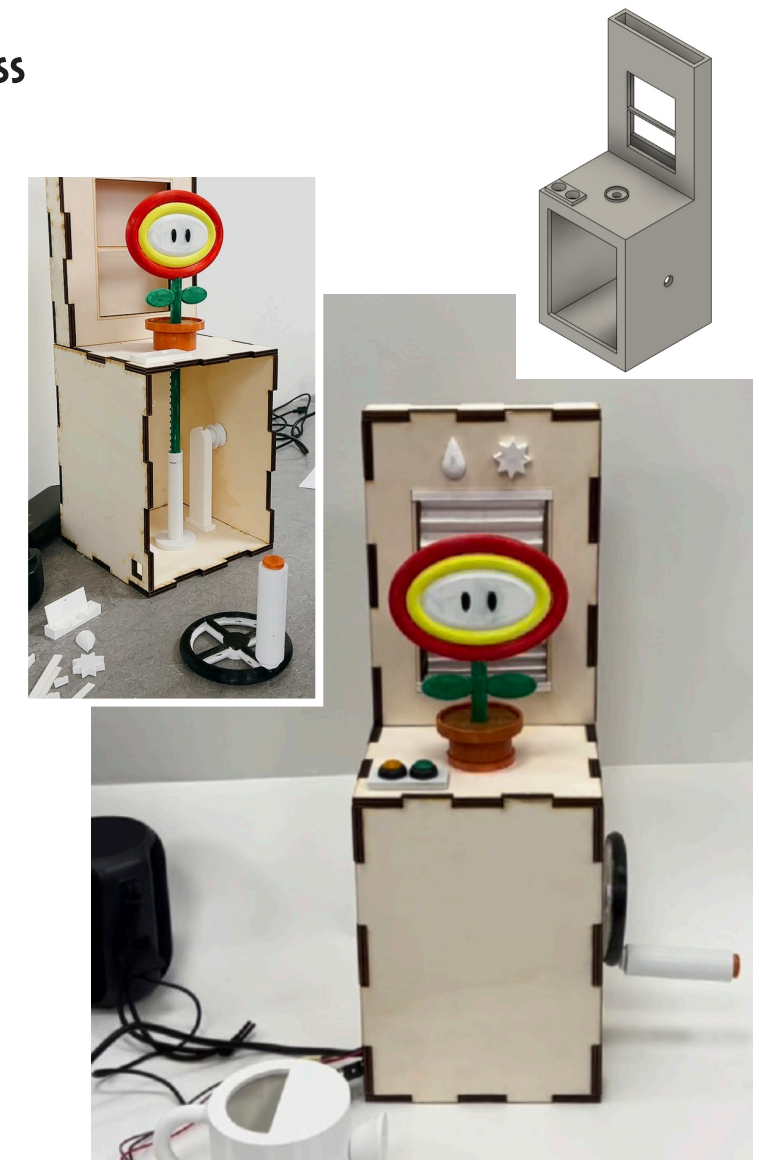
Step 2: The 3D printed parts were attached to the box and to other connecting parts.

Step 3: A paper blind and strings were connected through various components for the crank handle function.

Step 4: Housing parts for the flower pot and the rack, the motor, the ultrasonic sensor, and the buttons added stability.

Step 5: All electronics were assembled, attached into the box with required wires connected to the outside through a designated hole.

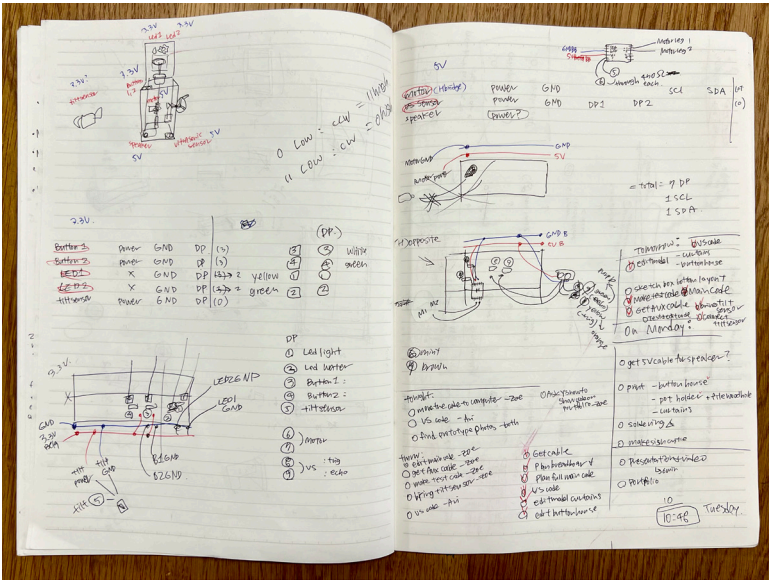
Step 6: The front plate of the box is closed.



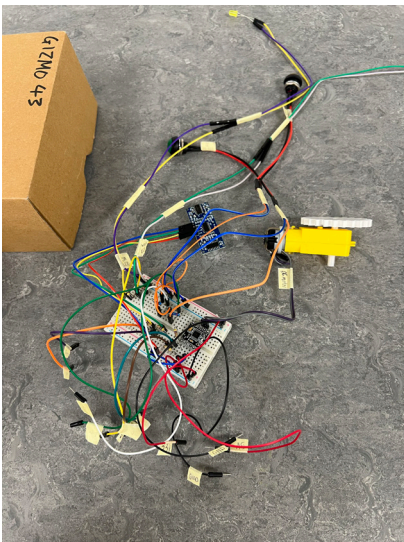
The Electronics

The Plan

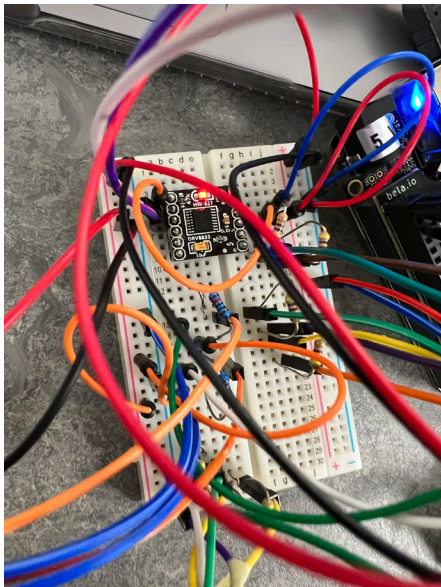
The plan outlines all connection and breadboard placings for both 3.3 V and 5V components (using separate halves of the breadboard except for the h-bridge), their pin number and wire colors.



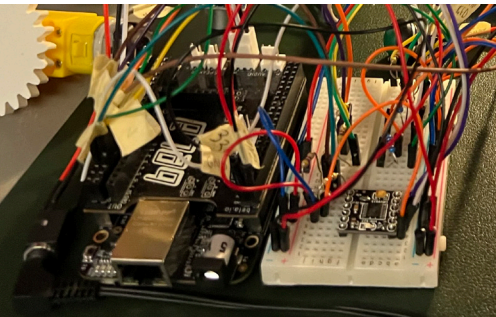
Real Life



(Top) Connection to the breadboard with all wires labelled as planned.



(Top) A close-up of the breadboard. Half is used by the 3.3V components, and the other half is used by the 5V components. Except for the h-board where its output pins take up part of the 3.3V side, but doesn't connect to power.



(Left) Connection to the bela board according to all the wire labels.

Structure

LEDs

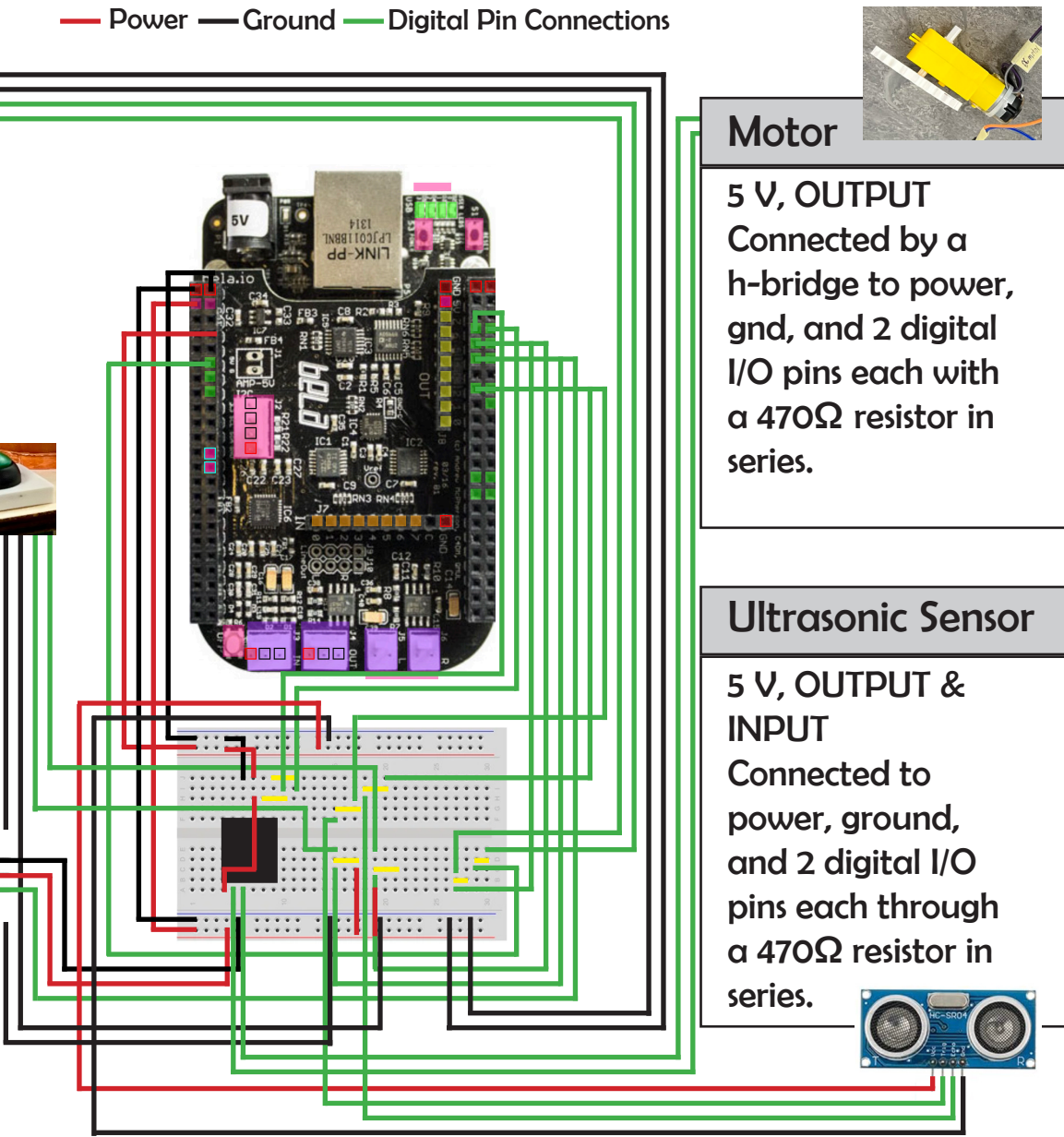
3.3 V, OUTPUT
Connected to power, gnd, and digital I/O pin in series with a 470Ω resistor.

Buttons

3.3 V, INPUT
Connected to power, gnd, and digital I/O pin with a 10kΩ resistor in parallel.

Tilt Sensor

3.3 V, INPUT
Connected to power, ground, and a digital I/O pin.



Motor

5 V, OUTPUT
Connected by a h-bridge to power, gnd, and 2 digital I/O pins each with a 470Ω resistor in series.

Ultrasonic Sensor

5 V, OUTPUT & INPUT
Connected to power, ground, and 2 digital I/O pins each through a 470Ω resistor in series.

Struggles & Solutions

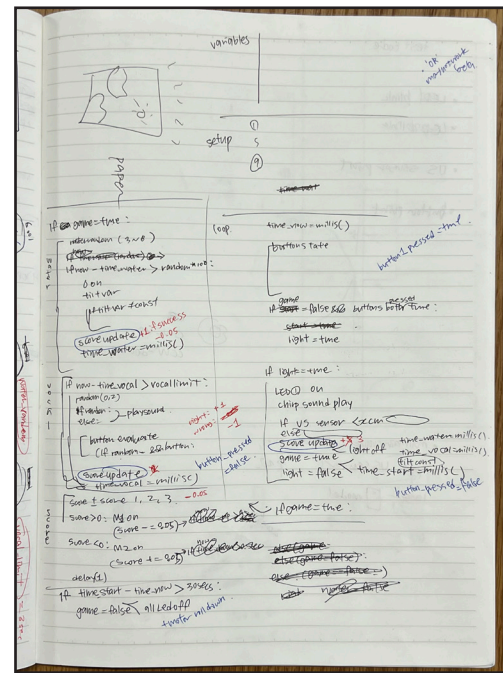
Whilst assembling the electronic components, some complications arose due to the unreliability and inconsistency in the DC motor which we resolved by replacing it with another DC motor.

Additionally, multiple iterations were carried out to find the appropriate duty cycle for the DC motor to withstand the weight of the fire flower and to rotate at a desirable speed.

When testing the Ultrasonic sensor, we had to identify where the example code from Bela fit in the render.cpp file and give access on BelaArduino.cpp for the necessary pins.

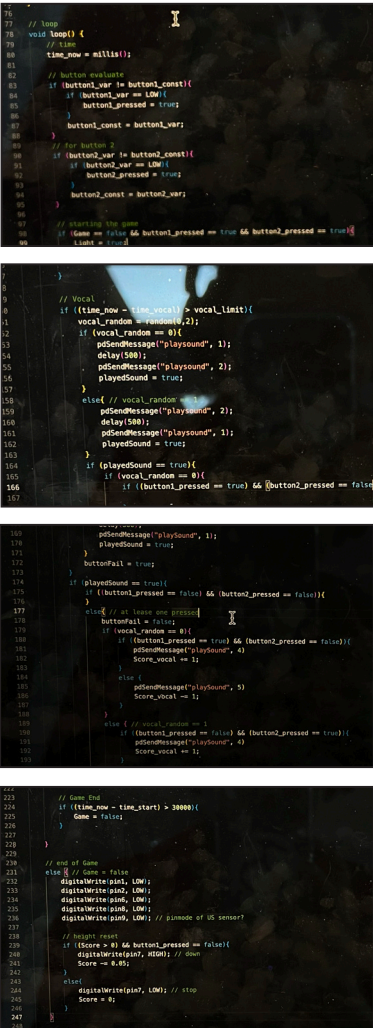
The Code

The Plan



Before starting the code, a plan was made outlining the structure of the code, including all the timings, loops, variables, boolean states and more.

The Process



These photos capture part of the progress of the initial coding process every 20 minutes.

Additional Codes

We had to modify render.cpp to utilize real-time readings of the ultrasonic sensor. We went through extensive efforts to solve this, but they were not successful. We presume this was due to the complexity of the structure of render.cpp, along with how we were missing additional files for the render.cpp that we couldn't find elsewhere. This was resolved with the help of the code of Group 8.

BelaArduino.cpp was slightly modified to allocate some of the digital I/O pins for render.cpp. This was resolved with the help of Group 3.

Main.pd was also modified to allow more than 5 sound files to play. This was resolved with the help of Group 27.

The ino.cpp Code Structure

pre-Setup

Include necessary external files.
Definition of all variables.
Variables for pin numbers.

void Setup

Setting all pin modes.
Loading all sound files.
Setting variables to base conditions for the initial Master Game.

void Loop (i.e. Master Game)

Update of Time_now.
Button state evaluation.
Light is true when both buttons are pressed.

Light == true

The “sunlight” LED turns on.
(User turns the crank handle) When the ultrasonic sensor reads value < 5cm, the LED turns off and chirping sound plays.
Variables are set to base conditions to allow multiple game plays.
Game is true. Light is false.

Game == true (i.e. Sub Game)

Water

Every 3~9 random seconds, the “water” LED is turned on.
If tilt sensor value is changed while the LED is on, it is a success.
Score of water updates accordingly.
A watering sound plays and LED is turned off.

Fertilizer

Every 5 seconds, a “increasing” sound and a “decreasing” sound plays in random order.
If you press the button for the “increasing” sound within the time limit, it is a success.
Score of Fertilizer updates accordingly.
A “success” sound or “failure” sound plays accordingly.

Score Update

The motor rotates to reflect the score change as the height of the flower.

Game Over

After 30 seconds, the game is over, a “game over” sound is played.
All output pins are set to LOW. Game over is set to false to allow multiple gameplays.

